Deliverable report

"Vittorio Polci": Mat: 257817, vittorio.polci@unitn.it, GitRepo: https://github.com/vittorio01/GPU-Computing-2025-257817

***Abstract*—The algorithm of sparse matrix-dense vector multiplication (SpMV) is an optimized version of the traditional matrix-vector multiplication, designed to handle cases where the matrix is organized in a low density of non-zero entries. This technique is particularly useful in scenarios involving very large matrices with a small amount of not null terms because it allows to reduce the used space for storing that large structures and, at the same time, optimizes memory accesses and reduces overhead during computational steps.**

This deliverable discusses about the implementation of different parallel implementation of the SpVM algorithm between CSR matrices and a general vector, with a final benchmark, which evaluates all solution toghether in order to identify the algorithm with the best behaviour.

***Index Terms*—Sparse Matrix, SpMV, CUDA, Parallelization, Storage Format**

I. Introduction

The main objective, when developing algorithms that elaborates large quantities of data, is to obtain a solution that:

1) Minimizes the execution time.
2) Minimizes the memory accesses during the computation of results.
3) Maintain as much as possible the performance with different inputs.

Because the SpMV multiplication should handle matrices with millions or billions of not null elements, to obtain good results is necessary to use parallelization techniques on GPUs rather than sequential solution for CPUs.

II. Problem Statement

In the SpVM multiplication we consider a matrix coded in a sparse format (in this case the CSR format) and an input vector that has the same size of its columns number.

Each elements of the resulting vector is computed by summing toghether the dot products of each row of the matrix with the input vector. At this point the main objectives are:

Algorithm 1 Algorithm for a standard matrix-vector product

```
1: procedure FUNCTION(matrix, vector, output)
2:     for row in matrix.rows do
3:         for element in row do
4:             output[row]
5:             = input[element] * output[element]
6:         end for
7:     end for
8: end procedure
```

1) Create a first sequential algorithm that performs the same operation on a sparse CSR matrix.
2) Parallelize in different ways the first CPU implementation with CUDA and compare them in terms of performances and resurces utilization.

A. Storage Format

The CSR for sparse matrices is a storage format that involves three different arrays to represent the information:

- The data array, that contains all not null terms of the original matrix.
- The column array, that for each element in the data vector specifies its column coordinate. For example, the element i in the data array has its respective column coordinate stored in $colArray[i]$
- The row array, which contains the start and the end pointers for each row to the data and column arrays. these two pair of pointers indicates which data are contained in that specific row.

At the end, the size of the data and column arrays depends on the number of not null elements and the size of the row array depends on the number of the rows in the original matrix.

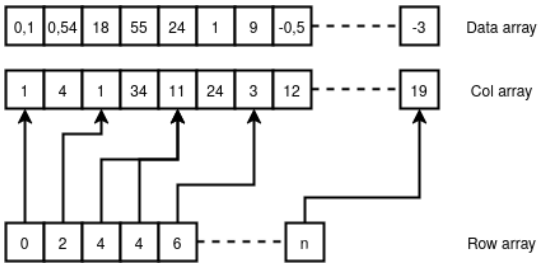

Fig. 1. Data organization in CSR structure

For example, all elements in contained in a row i are enumerated in the data array starting from the position stored in the row array $rowArray[i]$ to the position stored in the next cell $rowArray[i+1]$.

B. Parallelization

To improve the overall performance, the parallel implementation of the first algorithm are designed to be executed on Nvidia GPUs and programmed in CUDA. The CUDA language model is designed to run on different concurrent threads called CUDA cores, where each thread represents a computational unit located in the GPU.

In general, the workload can be splitted in different ways for a certain number of CUDA cores. Different organizations can provide more or less performance in terms of the execution time and troughput.

III. State of the Art

There already are multiple c++ libraries that performs CSR SpMV multiplications:

- cuSPARSE [1], the official library supported by Nvidia that can perform multiplication between matrices and vectors in different formats with CUDA.
- MAGMA-sparse [2], a linear algebra library that offers solution for multicore/multithreads algorithms that runs on CPUs and GPUs.
- Ginkgo [3], like MAGMA offers high-performance solution for multicore linear algebra algorithms for different platforms.

All listed libraries includes also shared memory optimizations, pipelining and other techniques to improve the performance of the algorithm.

The proposed algorithms in this derivable includes solutions that designed to performs only in the GPU's global memory.

IV. Methodology and Contributions

A. Sequential algorithm

There are not so much solutions to implement a sequential SpVM multiplication. The most intuitive and performant way is to scan the row vector and, using the stores pointers, perform the product and accumulate over a variable all the elements contained in each row.

Algorithm 2 Sequential version for the SpVM algorithm

Require: The input matrix structure $matrix$ that contains all arrays and sizes, the input vector $input$ structure, the number of not null elements $notNull$), the input vector structure (that contains the data array $dataArray$) and the output vector $output$ structure.

```
1:  procedure VMCSRMUL(output, matrix, input)
2:      for i in matrix.rowSize do
3:          startRow = matrix.rowArray[i]
4:          endRow = matrix.rowArray[i + 1]
5:          acc = 0
6:          for j in startRow, ..., endRow do
7:              col = matrix.colArray[j]
8:              acc+ =
9:              matrix.dataArray[j] * input.dataArray[col]
10:         end for
11:         output.dataArray[i] = acc
12:     end for
13: end procedure
```

The algorithm 2 scans and decodes each pair of start and end pointes for the data array and the column array of the input matrix. Using these two values a second loop first decode the column coordinate of each element in the specific row and then computes the product between the corresponding matrix and vector elements and accumulates the result in a value acc. After a row is processed by the second loop, the accumulated value is then stored in the second vector.

The memory access pattern of this solution permits coalescent accesses in all three arrays of the sparse matrix. The only aspect that limits the performance of the algorithm is that the accesses that gathers the input row elements depends on the values stored in the column array and they cannot be optimized without introducing overhead. input array should be readed,

B. First parallel implementation

A first parallel implementation of the algorithm 2 divides the workload between threads such a way that each thread processes one or more entire rows

The first operation that the algorithm performs is divide the workload for each thread. In particular:

1) The row array is splitted in equal subsection that will be assigned to different blocks of threads using the ceiling division system.
2) The obtained subsection are then divided in the same way in order to assign equal workload to each thread.

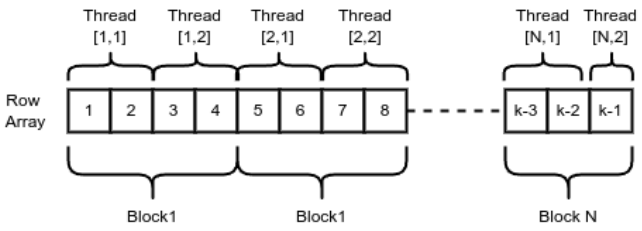

Fig. 2. Example of workload division for the first parallel implementation (N blocks of 2 threads)

After that procedure, each thread has his number of row to process $rowsPerThread$, the start position $threadStart$ and the end position $threadEnd$ in the row array.

Algorithm 3 First parallelization for the SpVM algorithm

Require: Same requirements in the sequential implementation but with all structures loaded in the global memory.

```
1:  procedure VMCSRMUL(output, matrix, input)
2:      threadStart, threadEnd  ▷ Ceiling division process
3:      for row in threadStart, ..., threadEnd do
4:          startRow = matrix.rowArray[i]
5:          endRow = matrix.rowArray[i + 1]
6:          acc = 0
7:          for j in startRow, ..., endRow do
8:              col = matrix.colArray[j]
9:              acc+ =
10:             matrix.dataArray[j] * input.dataArray[col]
11:         end for
12:         output.dataArray[row] = acc
13:     end for
14: end procedure
```

The algorithm 3 is the simplest implementation and has good performances for common sparse matrices because:

- For each thread the accesses are coalescent in all arrays of the matrix.

- Each thread works on the single rows independently and there is not concurrency at all.

C. Second parallel implementation

One problem of the algorithm 3 is that if there are threads that have much more more workload to perform, the computational time might be affected negatively. This is the case for matrices which have consecutive rows that contains much more not null elements compared to the others.

A solution that may mitigate this problem is to use a dynamic assignment of the rows to threads that are in the same block:

1) The row array is splitted in equal subsection for different blocks using again the ceiling division.
2) For each block a variable in the shared memory $pendingRow$ corresponds to the next row that is waiting for the assignment.
3) Each thread uses the function $atomicAdd(pendingRow, 1)$ to increment the value of $pendingRow$ and, at the same time, read its content and computes the corresponding row until they reach the end of their subsection.

Algorithm 4 Second parallelization for the SpVM algorithm

Require: Same requirements as the algorithm 3.

```
1:  procedure VMCSRMUL(output, matrix, input)
2:      blockStart, blockEnd                    ▷ Ceiling division
3:      pendingRow = 0                          ▷ (only thread 0);
4:      syncthreads()
5:      while true do
6:          pendingRow = atomicAdd(pendingRow, 1)
7:          +blockStart
8:          if pendingRow > blockEnd then return
9:          end if
10:         startRow = matrix.rowArray[pendingRow]
11:         endRow = matrix.rowArray[pendingRow + 1]
12:         acc = 0
13:         for j in startRow, ..., endRow do
14:             col = matrix.colArray[j]
15:             acc+ =
16:             matrix.dataArray[j] ∗ input.dataArray[col]
17:         end for
18:         output.dataArray[pendingRow] = acc
19:     end while
20: end procedure
```

The dynamic assignment for threads in the same block implemented in the algorithm 4 tries to mitigate the unbalanced workload introduced in some rows. In facts, if some thread spend too much time when computing a single row, other available threads may perform also rows that in the algorithm 3 should be reserved for others. Two limitation introduced with this system are:

- There is concurrency for the value $pendingRow$ that may limit the performance.
- There is a large probability that some threads do not elaborate rows that are coalescent in the matrix arrays.

D. Third parallel implementation

Another tentative to distribute the workload contained in the single rows is to parallelize the algorithm such that each row is assigned to single blocks instead of threads:

1) The row array is splitted for different blocks as usual.
2) For each row all threads does some multiplication (or not) of a certain number of elements and accumulates the result in the output vector using the function $atomicAdd()$.

Algorithm 5 Second parallelization for the SpVM algorithm

Require: Same requirements as the algorithm 3.

```
1:  procedure VMCSRMUL(output, matrix, input)
2:      blockStart, blockEnd                    ▷ Ceiling division
3:      for row in blockStart, ..., blockEnd do
4:          startRow = matrix.rowArray[row]
5:          endRow = matrix.rowArray[row + 1]
6:          elements = endRow − startRow
7:          if elements < blockSize.x then
8:              col = matrix.colArray[j]
9:              atomicAdd(
10:             output.dataArray[row],
11:             matrix.dataArray[threadIdx.x] ∗
12:             input.dataArray[col]
13:             )
14:         else
15:             threadStart,
16:             threadEnd                       ▷ Ceiling division on row
17:             acc = 0
18:             for j in threadStart, ..., threadEnd do
19:                 col = matrix.colArray[j]
20:                 acc+ = matrix.dataArray[j] ∗
21:                 input.dataArray[col]
22:             end for
23:             atomicAdd(
24:             output.dataArray[row], acc
25:             )
26:         end if
27:     end for
28: end procedure
```

The algorithm 5 distributes the workload of a row on blocks of threads instead single threads but, without using shared memory optimization, this creates more overhead because:

- If the block size is large the function $atomicAdd()$ creates a barrier that limits too much the performance because it must guarantee atomic accumulation on the output vector.
- The first threads in a block are overused for all rows but the last threads are used only if the number of rows are large enough to require them.

V. System Description and Experimental Set-up

A. Dataset description

A dataset composed by 6 matrices is used for testing and benchmark all four algorithms. These matrices containes a number of not null elements between 1 and 1,5 million of not null elements. These matrices have different organization of the elements and dispersion that challenges the algorithms in different ways:

- Linux_call_graph represents one of the worst cases because it has a large amount of not null elements that are distribuited on all original matrix.
- Nemeth24 represents one of the better cases because it has a low amount of elements that are concentrated on the diagonal.

The used dataset is obtained from the SuiteSparse Matrix Collection [4], a website that contains sample matrices which can be used for benchmarking algorithms.

Dataset	Dimension (R x C)	Non-zero entries	Type
dbir2 [5]	18,906 x 45,887	1,158,159	Unbalanced rows
ex11 [6]	16,614 x 16,614	1,096,948	Diagonally distributed (low density)
language [7]	399,130 x 199,130	1,216,334	Upper triangular
Linux_call_graph [8]	324,085 x 324,085	1,208,908	Dense matrix (worst case)
nemeth24 [9]	9,506 x 9,506	1,506,550	Diagonally distributed (best case)
twotone [10]	120,750 x 120,750	1,206,265	Diagonally distributed (moderate dispersion)

TABLE I
Structural patterns of the benchmark sparse matrices.

B. System Description

The system used for the execution of the benchmarks is uses CUDA 12.3.2, GCC 11.5.0, and is composed by a Intel(R) Xeon(R) Silver 4214 CPU and A Nvidia A30 GPU.

Characteristic	Value	Device
Global memory	24 GB	
Theoretical BW	933 GB/s	
CUDA Cores	10752	**GPU: Nvidia A30**
SM Number	108	
FP32 Performance	10.3 TFLOPS	
Cores number	12	
Threads number	24	**CPU: Intel(R) Xeon(R) Silver 4214**
Clock speed	2.2Ghz	

TABLE II
Benchmark system specs

C. Experimental Set-up

The performance is measured using the execution time elapsed measured with the C function $getTimeOfDay()$ for the sequential algorithm and $cudaEventRecord$ functions for CUDA algorithms.

Each algorithm is executed 25 times (5 warmup cycles and 20 iterations) and from the execution times (in milliseconds) is performed the geometric mean to remove eventual outlayers.

From the average execution time is then performed the effective bandwidth and the FLOP/s.

$$effBW(GB/s) = \frac{readedBytes + writtenBytes}{executionTime(ms)} * 10^{-6} \quad (1)$$

$$GFLOPS/s = \frac{floatingPointOp}{executionTime(ms)} * 10^{-6} \quad (2)$$

VI. Experimental Results

Present and discuss results. Include plots and tables when required (like Figure 3). Do not simply describe the figures; criticise the achieved results by underlining how they confirm/differ from the expected outcomes described in Section IV.

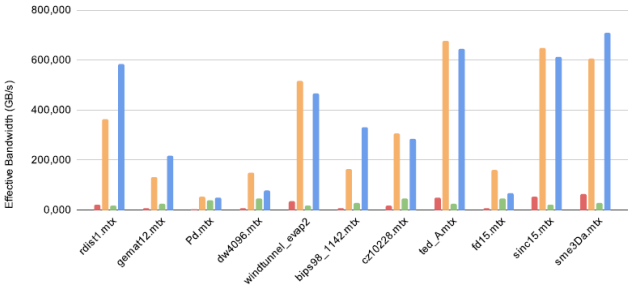

Fig. 3. Caption

VII. Conclusions

[max 200 words]
Summarize findings and future work ...

References

[1] NVIDIA. cusparse. [Online]. Available: https://developer.nvidia.com/cusparse
[2] u. o. T. Innovative Computer Laboratory. Magma. [Online]. Available: https://icl.utk.edu/magma/
[3] Ginkgo. [Online]. Available: https://ginkgo-project.github.io
[4] U. of Florida. Suitesparse matrix collection. [Online]. Available: https://sparse.tamu.edu/
[5] Meszaros. dbir2 matrix. [Online]. Available: https://sparse.tamu.edu/Meszaros/dbir2
[6] FIDAP. ex11 matrix. [Online]. Available: https://sparse.tamu.edu/FIDAP/ex11
[7] Tromble. language matrix. [Online]. Available: https://sparse.tamu.edu/Tromble/language
[8] Sorensen. Linux call graph matrix. [Online]. Available: https://sparse.tamu.edu/Sorensen/Linux_call_graph
[9] Nemeth. nemet24 matrix. [Online]. Available: https://sparse.tamu.edu/Nemeth/nemeth24
[10] ATandT. twotone matrix. [Online]. Available: https://sparse.tamu.edu/ATandT/twotone